Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:**	**Enrolment No:92301733042**

Aim: To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)

IDE: Google Colab

Theory:

To preprocess your text simply means to bring your text into a form that is predictable and analyzable for your task. A task here is a combination of approach and domain. Machine Learning needs data in the numeric form. We basically used encoding technique (Bag Of Word, Bi-gram, n-gram, TF-IDF, Word2Vec) to encode text into numeric vector. But before encoding we first need to clean the text data and this process to prepare (or clean) text data before encoding is called text preprocessing, this is the very first step to solve the NLP problems.

Tokenization:
Tokenization is about splitting strings of text into smaller pieces, or "tokens". Paragraphs can be tokenized into sentences and sentences can be tokenized into words.

Filtration:
Similarly, if we are doing simple word counts, or trying to visualize our text with a word cloud, stopwords are sonic of the most frequently occurring words but don't really tell us anything. We're often better off tossing the stopwords out of the text. By checking the Filter Stopwords option in the Text Pre-processing tool, you can automatically filter these words out.

Stemming:
Stemming is the process of reducing inflection in words (e.g. troubled, troubles) to their root form (e.g. trouble). The "root" in this case may not be a real root word, but just a canonical form of the original word.

Stemming uses a crude heuristic process that chops off the ends of words in the hope of correctly actually be converted to troublinstead of trouble because the ends were just chopped off (ughh, how crude!). There are different algorithms for stemming. The most common algorithm, which is also known to be empirically effective for English, is Porters Algorithm. Here is an example of stemming in action with Porter Stemmer:

original word stemmed words

0	connect	connect
'I	connected	connect
2	connection	connect
3	connections	connect
4	connects	connect

Subject: Artificial Intelligence (01CT0616)
Experiment No: 6

Aim: To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)

Date: **Enrolment No:92301733043**

Stopword Removal

Stop words are a set of commonly used words in a language. Examples of stop words in English are "a", ‘‘the", "is", "are" and etc. The intuition behind using stop words is that, by removing low information words from text, we can focus on the important words instead.

For example, in the context of a search system, if your search query is 'what is text preprocessing?", you want the search system to focus on surfacing documents that talk about text preprocessing over documents that talk about what is. This can be done by preventing all words from your stop word list from being analyzed. Stop words are commonly applied in search systems, text classification applications, topic modeling, topic extraction and others. Stop word removal, while effective in search and topic extraction systems, showed to be non-critical in classification systems. However, it does help reduce the number of features in consideration which helps keep your models decently sized

Program (Code):

```
text="""Hello my name in Ansh Raythatha,currently i am studing at Marwadi University
located at , Rajkot, Gujarat,lectures timings are 8:00 AM to 1:20 PM , from Monday to
Friday"""
#tokenization text.split()
    # i like playing cricket -4 words
    # i like play ing cricket -5 tokens

    import re #Regex: regular exspreesion import
    string
    string.punctuation

    #nlp    library
    import  spacy
    #gensim,nltk
```

nlp=spacy.load('en_core_web_sm')
text text[5] doc=nlp(text) doc
doc[11]

#NER => Name Entity Recognition for
token in doc.ents:
print(token.text,token.label_)

#post tags for
token in doc:
print(token.text,"->",token.pos_) import
nltk

#stopwords
from nltk.corpus import stopwords nltk.download('stopwords')
set(stopwords.words('english'))

#Stemming vs lemmatization from nltk.stem.porter import *
p_stem=PorterStemmer() text=""" I study the subject name AI and then meeting
Mr.Rohan tommarrow in the meeting room""" for word in text.split():
print(word,"->",p_stem.stem(word))

#lemmatization for token in nlp(text):
print(token.text,"->",token.pos_,"->",token.lemma_)

<u>Results:</u>

Subject: Artificial Intelligence (01CT0616) **Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)


```
Ansh Raythatha PERSON
Marwadi University ORG
Rajkot GPE
Gujarat GPE
8:00 AM TIME
1:20 PM TIME
Monday to Friday DATE
```

```
{'a',
 'about',
 'above',
 'after',
 'again',
 'against',
 'ain',
 'all',
 'am',
 'an',
 'and',
 'any',
 'are',
 'aren',
 "aren't",
 'as',
 'at',
 'be',
 'because',
 'been',
 'before',
 'being',
 'below',
 'between',
 'both',
 'but',
 'by',
 'can',
 'couldn',
 "couldn't",
 'd',
 'did',
 'didn',
 "didn't",
 'do',
 'does',
 'doesn',
 "doesn't",
 'doing',
 'don',
 "don't",
 'down',
 'during',
 'each',
 'few',
 'for',
 'from',
 'further',
 'had',
```

```
I -> i
study -> studi
the -> the
subject -> subject
name -> name
AI -> ai
and -> and
then -> then
meeting -> meet
Mr.Rohan -> mr.rohan
tommarrow -> tommarrow
in -> in
the -> the
meeting -> meet
room -> room
```

```
 -> SPACE ->
I -> PRON -> I
study -> VERB -> study
the -> DET -> the
subject -> ADJ -> subject
name -> NOUN -> name
AI -> PROPN -> AI
and -> CCONJ -> and
then -> ADV -> then
meeting -> VERB -> meet
Mr. -> PROPN -> Mr.
Rohan -> PROPN -> Rohan
tommarrow -> VERB -> tommarrow
in -> ADP -> in

 -> SPACE ->

the -> DET -> the
meeting -> NOUN -> meeting
room -> NOUN -> room
```

Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:**	**Enrolment No:92301733042**

Observation:

From this experiment, we can clearly understand that text preprocessing is a crucial first step in any NLP task. Raw text data is usually unstructured, inconsistent, and contains unnecessary words or symbols. After applying preprocessing techniques, the text becomes more structured, clean, and suitable for further analysis or machine learning modeles

Post Lab Exercise:

1. Take your own document of 10 sentences and perform the same tasks. Attach code and screenshot of your output. **CODE:**

```
# Install required libraries import nltk nltk.download('punkt')
nltk.download('punkt_tab') nltk.download('stopwords')
nltk.download('wordnet') from nltk.tokenize import
word_tokenize, sent_tokenize from nltk.corpus import
stopwords from nltk.stem import PorterStemmer,
WordNetLemmatizer


# Sample document text = """Natural language processing
is a fascinating field.
Machines can understand human language with proper training.
Text preprocessing is the first step in NLP.
Tokenization splits text into words or sentences.
Stopwords are removed to improve efficiency.
Stemming reduces words to their base form. Lemmatization
gives meaningful root words.
Data cleaning improves model performance.
Python provides many libraries for NLP tasks.
Learning NLP is useful for many real-world applications."""


# Tokenization sent_tokens =
sent_tokenize(text) word_tokens =
word_tokenize(text) print("Sentence
Tokens:\n", sent_tokens) print("\nWord
Tokens:\n", word_tokens)
```

<table>
<tr><td rowspan="1">Marwadi University</td><td colspan="2">Marwadi University
Faculty of Technology
Department of Information and Communication Technology</td></tr>
<tr><td>Subject: Artificial Intelligence (01CT0616)</td><td colspan="2">Aim: To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)</td></tr>
<tr><td>Experiment No: 6</td><td>Date:</td><td>Enrolment No:92301733042</td></tr>
</table>

OUTPUT:

```
Sentence Tokens:
 ['Natural language processing is a fascinating field.', 'Machines can understand human language with proper training.', 'Text preprocessing is the first step in NLP.', 'Tokenization spl

Word Tokens:
 ['Natural', 'language', 'processing', 'is', 'a', 'fascinating', 'field', '.', 'Machines', 'can', 'understand', 'human', 'language', 'with', 'proper', 'training', '.', 'Text', 'preproces

Filtered Words:
 ['Natural', 'language', 'processing', 'fascinating', 'field', '.', 'Machines', 'understand', 'human', 'language', 'proper', 'training', '.', 'Text', 'preprocessing', 'first', 'step', 'N

Stemmed Words:
 ['natur', 'languag', 'process', 'fascin', 'field', '.', 'machin', 'understand', 'human', 'languag', 'proper', 'train', '.', 'text', 'preprocess', 'first', 'step', 'nlp', '.', 'token', '

Lemmatized Words:
 ['Natural', 'language', 'processing', 'fascinating', 'field', '.', 'Machines', 'understand', 'human', 'language', 'proper', 'training', '.', 'Text', 'preprocessing', 'first', 'step', 'N
```

```
nto words or sentences.', 'Stopwords are removed to improve efficiency.', 'Stemming reduces words to their base form.', 'Lemmatization gives meaningful root words.', 'Data cleaning improv

', 'the', 'first', 'step', 'in', 'NLP', '.', 'Tokenization', 'splits', 'text', 'into', 'words', 'or', 'sentences', '.', 'Stopwords', 'are', 'removed', 'to', 'improve', 'efficiency', '.',

'Tokenization', 'splits', 'text', 'words', 'sentences', '.', 'Stopwords', 'removed', 'improve', 'efficiency', '.', 'Stemming', 'reduces', 'words', 'base', 'form', '.', 'Lemmatization', 'g

ext', 'word', 'sentenc', '.', 'stopword', 'remov', 'improv', 'effici', '.', 'stem', 'reduc', 'word', 'base', 'form', '.', 'lemmat', 'give', 'meaning', 'root', 'word', '.', 'data', 'clean'

'Tokenization', 'split', 'text', 'word', 'sentence', '.', 'Stopwords', 'removed', 'improve', 'efficiency', '.', 'Stemming', 'reduces', 'word', 'base', 'form', '.', 'Lemmatization', 'give'
```

2. Write your observation for stemming and lemmatization you obtained for the sentences.

    # Stopword Removal
    stop_words = set(stopwords.words("english")) filtered_words = [w for w in word_tokens if w.lower() not in stop_words] print("\nFiltered Words:\n", filtered_words)

    # Stemming ps = PorterStemmer() stemmed_words = [ps.stem(w) for w in filtered_words] print("\nStemmed Words:\n", stemmed_words)

    # Lemmatization lemmatizer = WordNetLemmatizer() lemmatized_words = [lemmatizer.lemmatize(w) for w in filtered_words] print("\nLemmatized Words:\n", lemmatized_words)

Stemming Observation

- Stemming removes suffixes using simple rules.
- Words are reduced to **non-meaningful roots** sometimes.
- Example:
    - *processing → process*
    - *fascinating → fascin* (not a proper word)
- It is **fast but less accurate**.

Lemmatization Observation

- Produces **meaningful base words** (dictionary form).
- Uses vocabulary and morphological analysis.
- Example:
    - *processing → processing* (or *process* depending on POS)
    - *better → good* (if POS applied)
- It is **slower but more accurate**.